

BİL 468 / BİL 587 – PROJE SON RAPORU

Erişim Bilgileri

Öğrenci ID ve İsmi: 231101018 Baturalp İskenderoğlu

Öğrenci ID ve İsmi: 231101046 Efe Görkem Akkanat

Grup No: 1

Grup Gmail Adresi: baturalpiskenderoglu0@gmail.com

Orijinal Veri Seti Adı: Drone Detection

Orijinal Veri Seti Bağlantısı: <https://universe.roboflow.com/aatish-kumar-sahu-57emd/drone-detection-1ghph>

Proje Çalışmasının Yüklendiği Google Drive Bağlantısı:

<https://drive.google.com/drive/folders/1jBMcBOWnol4iAeXokRELNyMksjStSz9q?usp=sharing>

Çalıştırılacak Dosyalar ve Açıklamaları:

Yontem1 klasörü içerisinde:

config.py: Hiperparametrelerin tutulduğu dosya.

preprocess.py: Ön işlemeyi yapan dosya.

dataset.py: Modele verilecek DroneDataset sınıfını barındırır.

model.py: Modeli barındırır.

utils.py IoU hesabı, loss hesabı, Non-maximum suppression gibi işlemler için fonksiyonları barındırır.

train.py: Eğitim ve geçerlemenin yapıldığı dosya.

Demo.ipynb: Modelin çalışmasının örneklerle sergilendiği demo dosyası.

Evaluation.ipynb: Başarım ölçümlerinin yapıldığı dosya.

Yontem2 klasörü içerisinde:

Transfer_learning_model.py: Yöntem 2'de kullanılan modeli (TransferLearningModel sınıfını) barındırır. Loss hesabının yapıldığı sınıf olan DroneLoss'u barındırmaktadır. Eğitim methodu olan train() methodunu barındırır. Inference methodları, başarım metriklerini hesaplayan metotlar ve geçerleme metotlarını barındırmaktadır.

Validation.ipynb: Geçerlemenin yapıldığı ve tüm deneylerin yapıp sonuçlarıyla sunulduğu dosyadır.

Test.ipynb: Test sonuçları ve model demosunun sunulduğu dosya.

DroneDataset.py: Modele verilen DroneDataset sınıfını barındırır. Burada görüntü belirli normalizasyonlardan geçirilir, label'lar düzenlenir. Aynı zamanda padding yapan fonksiyonu barındırır.

preprocess.py: Ön işleminin yapıldığı ve yeni datasetin oluşturulduğu dosya.

Experiment_<deney_no>.json: Bu dosyalar validation.ipynb'de yapılan deneylerin sonuçlarını tutar.

Experiment10_model: En iyi modelin .pth dosyasını barındıran klasör (içerisinde yer alan mode_0.pt en iyi modele aittir.)

Experiment11_model: 11. Deneyde eğitilen modelin .pth (model ağırlıklarının) dosyasını barındıran klasör. (içerisinde yer alan mode_0.pt 11. Deneyde eğitilen modele aittir.)

1. Proje Konusu

Bu projede, çeşitli boyut, şekil ve renklerde olabilen drone'ların çeşitli boyutlarda veri boyutları barındıran 18 bin sample'lık Drone Detection veriseti kullanılarak aşağıda detayları açıklanan 2 farklı yöntem ile tespiti hedeflenmektedir.

2. Literatür Taraması

FPGA-Based CNN for Real-Time UAV Tracking and Detection

Bu makalede CNN tabanlı Drone (UAV) tespiti yapan sinir ağı mimarisinin FPGA üzerinde çalışması (inference) için optimize edilmiş şekilde tasarlanmasından bahsedilmektedir. Temel problem, FPGA'in floating point işlemleri yaparken kaynak kısıtlarına takılmasıdır. Makalede bu sorunun çözümü için quantization uygulanması ve lineer aktivasyon fonksiyonları kullanılması şeklinde optimizasyonlar yapılmıştır. Bizim yöntemimizden farklı olarak bu çalışmada yüksek donanım kısıtı bulunmaktadır. Biz, kendi yöntemimizde katı donanım kısıtları olmaması sebebiyle quantization yapmayacağız. Makalede 1000 görüntü içeren veri setlerini zenginleştirmek için aynı görüntünün farklı rotasyon'larını alarak yeni görüntülerle datasetlerini zenginleştirdiklerinden bahsetmektedir. Bizim veri setimizin yeterince büyük olması sebebiyle kendi yöntemimizde bu tür bir işleme başvurmayacağız. Makalenin devamında drone takibi için ardışık video frame'lerinin birbirinden çıkarılması sonucu elde edilen residual frame elde edilmesi şeklinde yapılan ön işlemlerden bahsedilmiştir. Makalede aynı zamanda residual image sayesinde ardışık frame'lerdeki hareketli nesnelerin tracker tarafında tespit edilebileceği ve bu tracker'ın tespit ettiği değişimin CNN tarafından sınıflandırılabilceği tarzında yöntemlerden bahsedilmiştir. Bizim projemizde obje takibi olmaması ve video üzerinde çalışılmaması sebebiyle makalenin bu yönü projemiz ile ayrılmaktadır.[1]

CNN-Based UAV Detection and Classification Using Sensor Fusion

Bu makalede drone tespiti için görüntü verisinin, "range-Doppler haritası" formundaki radar verisinin ve spektrogram formundaki ses verisinin her biri ayrı bir CNN'e verilmiş, ardından her bir CNN'in olasılık çıktısı logistic regression modeline verilerek drone olup olmadığına dair bir çıktı üretilmiş, drone tespit edildiği takdirde tespit modeli ile aynı mimariye sahip olan sınıflandırma için eğitilmiş ayrı bir sinir ağında aynı şekilde 3 ayrı CNN çıktıları logistic regression'a verilerek sınıflandırma yapılmaktadır. Kullanılan CNN'ler de transfer learning yapılmıştır. Bu makalede bahsi geçen projenin, bizim projemizden farkı ise bizim veri setimizin radar ve ses verisi içermemesi bununla beraber bizim projemizde tek sınıf olması ve makaledekinden farklı olarak belirlenen drone'un resim içindeki konumunun belirtilmesidir.[2] Bu yöntem 3 farklı çeşit veri ile başarımın artırdığını göstermesine rağmen gerçek hayatta aynı anda hem görüntü hem radar hem de ses verisinin aynı anda toplanması beklenmeyeceğinde veri toplanması ve uygulanması açısından zorludur.

Toward Efficient UAV-Based Small Object Detection:

Bu makalede İHA'lerden elde edilen görüntülerdeki küçük hedeflerin tespiti sırasında ölçek değişimleri ve occlusion sorunlarına odaklanılmış, başarılı aynı zamanda hafif bir model önerilmiştir. Makalede çift taraflı evrişim ve çapraz seviye özellik ilişkilendirilmesi (SPPL) kullanılarak küçük hedeflerin düşük çözünürlükte daha iyi öğrenilmesi sağlanmıştır. Ayrıca bounding box hassasiyetini arttırmak için WIoU adında yeni bir loss function önerilmiştir. Bizim modellerimizden farklı olarak YOLO tabanlı bir geliştirmedir. Bizim projemiz adına WIoU loss function kullanımı değerlendirilebilir.[3]

A Small-Object Detection Model Based on Improved YOLOv8s for UAV Image Scenarios:

Bu makalede yüksek irtifadan çekilen İHA görüntülerindeki arka plan gürültüsünü (background noise) ve küçük nesne yoğunluğu sorunlarını aşabilmek için paralel çok ölçekli özellik çıkarma modülü (PMSE) geliştirilmiştir. Makale farklı receptive fieldlar üzerinden ağırlıklar üretilen küçük hedeflerin daha kolay ayırt edilmesini sağlayan bir yapı sunmuş ve sınıf dengesizliğini gidermek amacıyla WIoU loss functionunu kullanmıştır. Bu makaledeki WIoU tercih sebepleri bizim yöntem 1'de Focal Loss tercih sebeplerimizle benzerdir bu sebeple incelemeye değer öneriler içerir.[4]

Maritime Small Object Detection from UAVs using Deep Learning with Altitude-Aware Dynamic Tiling:

Bu makalede, İHA'lar ile yüksek irtifa resimlerinde yine [3] benzeri az pixel kaplayan nesne tespit problemine odaklanılmış ve "Slicing Aided Hyper Inference (SAHI)" isimli bir tiling/sliding window yöntemi önerilmiştir. Makalede bizim 1. Yöntemimize benzer yüksek çözünürlüklü görüntüler overlapping yamalara

bölünmüş yapılan tahminler NMS ile ana resimde birleştirilmiştir. Ancak burada önerilen yöntem 1. yöntemimizdeki gibi sabit bir 224x224 window aksine uçuş irtifasına bağlı dinamik pencere boyutları kullanan adaptif bir yöntemdir. Gelecek geliştirmeleri arasındaki anchor box sayısı artırılması adına incelenebilir.[5]

3. Veri Seti ve Değerlendirme Metrikleri

Veri seti içerisinde drone içermeyen, tek drone içeren, birden fazla drone içeren örnekler bulunmaktadır. Bu örneklerden bazılarında drone, görüntünün büyük bir kısmını kaplamakta iken bazı görüntülerde küçük bir yer kaplamaktadır. Veri seti içerisinde drone olmayan fakat drone'u andırabilecek uçak, kuş, balon gibi farklı objelerin de resmi bulunmaktadır.

Veri seti aldığımız kaynakta 3 parçaya ayrılmış haldeydi. Biz de aynı ayrımı koruduk.

Eğitim veri seti: 11.577 görüntü

Geçerleme veri seti: 3.952 görüntü

Test veri seti: 2.597 görüntü

Ön işleme süreçleri her yöntem için ayrı yapılmıştır ve her yöntem için kendi başlığı altında anlatılmıştır.

4. Yöntem 1

Bu yöntemde önceden eğitilmiş hiçbir modeli kullanmadan sıfırdan kendi verisetimiz üzerinde optimize edilmiş bir CNN/Dense Head mimarisi geliştirilmiştir. Veri setindeki görsellerindeki devasa boyut farklılıkları ve bazı drone örneklerinin çok küçük bazılarının çok büyük olması sebebiyle training, preprocessing ile sadece drone bboxlarından kesilip kareye random jitter uygulanarak oturtulmuş 224x224 boyuta scale edilmiş positive samplelar ve bbox dışında kalan kısımlardan yine preprocessing aşamasında rastgele kesilmiş 224x224 scale edilmiş negatif samplelar üzerinden gerçekleştirilmiştir.

Training sırasında sadece kare 224x224 blokta random augmentation jitter ile ortada drone tespit etmeyi öğrenmiş model, inference aşamasında bboxları bilinmeyen test verisini tanıyabilmesi adına inference aşamasında [0.5, 1, 1.5]'lik image pyramid tüm test verisine uygulanmış model bu image pyramid kullanılarak scale edilmiş test görüntülerinin her birinin üzerinde gezen 224x224'lük window üzerinden tahmin üretmiştir.

Tüm bu merkezi drone tanıma modeli ve image pyramid pipeline'ının işlevli çalışabilmesi adına training ve validation veri setleri ağır bir preprocessing aşamasından geçirilmiştir. Bu aşamanın detayları aşağıdadır.

Preprocessing:

- 1) Veri setindeki içerisinde drone bulunduran tüm bounding box koordinatları ilgili resimler üzerinden dış tarafına random ince bir padding ve random x, y jitter uygulanarak kesilmiştir.
- 2) Kesilen bu farklı boyutlardaki drone karoları 224x224 lük window size ile eşit boyuta gelecek şekilde yeniden boyutlandırılarak pozitif örneklerimiz elde edilmiştir.
- 3) Ardından işlenmemiş veriler üzerinden pozitif samplelar ile aynı miktarda rastgele windowlar kesilmiştir.
- 4) Bu windowların her biri içinden kesildiği resimdeki Drone bboxlarıyla IoU karşılaştırılmasına sokulmuş ve eğer [config.py](#)'da bulunan NEG_IOW_THRESHOLD parametresinden daha büyük yüzde ile drone bbox'uyla kesişiyor ise negatif sample listesinden çıkartılmıştır.
- 5) Kalan kesikler drone içermeyen negatif samplelar 0 confidence ile label'lanarak processed klasörü altında pozitif samplelar ile beraber kaydedilmiştir.

Model Tasarımı ve Mimari Detaylar:

Model mimarisi özellik çıkarıcı ve tespit katmanı(Dense Head) olmak üzere 2 ana bölümden oluşmaktadır. İki ana bölümdeki aşağıda sıralanan parametrelerin her biri hiperparametre optimizasyonuna zaman ve donanım el verdiğince tabi tutulmuş elde edilen imkanlar doğrultusunda açıklanacak mimari f1 skoru kullanılarak yapılmış optimizasyon sonucunda en başarılı sonucu vermiştir.

Denenen Hiperparametre Kombinasyon havuzu:

```
TRAIN = {
    "MAX_EPOCHS": 100,
    "EARLY_STOPPING_METRIC": "f1",
    "PATIENCE": [6, 12],
    "BATCH_SIZE": [32, 64],
    "LEARNING_RATE": [1e-4, 1e-5, 5e-6],
    "OPTIMIZER": ["adam", "rms-prop"],
    "ACTIVATION": ["leaky_relu", "gelu"],
    "NUM_CONV_BLOCKS": [4, 5],
    "BASE_FILTERS": [16, 32],
    "DENSE_NEURONS": [(128, 64), (64, 32)],
    "DROPOUT_RATE": [0.3, 0.5]
}
```

En yüksek F1 skoru veren kombinasyon:

```
{
    "lr": 0.0001,
    "batch": 32,
    "optimizer": "adam",
    "blocks": 5,
    "base_filters": 32,
    "dense": [
        128,
        64
    ],
    "dropout": 0.3,
    "activation": "leaky_relu"
}
```

Convolution katmanının hiper parametre havuzu belirlenmesinde aşağıdaki kriterler rol oynamıştır:

224x224 girdi boyutu kullanıldığından ve convolution katmanları arasına max pooling layer eklendiğinden dolayı 5 convolution block sonrasında çıktı boyutu 7x7 ye kadar düşmektedir bu noktadan sonra max pooling uygulanması 7, 2'ye bölünmediği ve 3x3 olarak aşağı yuvarlanacağı için önerilmemektedir bu sebeple max 5 convolution block denenmiştir.

Filtre sayısı her katmanda 2 katına çıkartmak standart olarak tercih edildiğinden yalnızca başlangıç filtre sayısı hiperparametre optimizasyonuna dahil edilmiştir. Base Filters parametre sayısını üstel arttırdığı ve eğitimi zorlaştırdığı için maximum 32 filtreli başlangıç denenmiştir.

Çıktı yapısı: Ağın çıktı katmanı 5 nöron içermektedir. İlk nöron 224x224 lük patchde drone bulunma ihtimalini(confidence) ifade ederken diğer 4 nöron yama içerisindeki merkez ve boyut koordinat bilgilerini tahmin etmek için kullanılmıştır.(x,y,w,h)

Multi-task Loss Function: Model aynı anda confidence skoru ve regresyon koordinat tahmini yürüttüğü için bir Multi-task Kayıp fonksiyonu üzerinden eğitimi gerçekleştirilmiştir.

Model kayan pencere kullanımında çok fazla arka plan patchi görmektedir. Bu durum modelin her şeye arka plan diyerek tembellişmesine yol açmaktadır.

Bu durumun önüne geçmek adına standart BCE(Binary Cross-Entropy) yerine BCE temelli bir Focal Loss function kullanılmıştır. Bu fonksiyonun kullanımındaki fikir (1-p) çarpanı sayesinde emin bir şekilde yapılan yanlışlara ağır loss uygulanmasıdır. Bunun yanı sıra pos_weights parametresi train öncesinde initial hesaplanarak sınıflar arası ağırlıklandırmayı loss hesabına dahil ederek bu durumun olabildiğince önüne geçer.

Regression kısmında ise son 4 çıktı nöronu bounding box tahmini yapar. Bu 4 çıktı klasik MSE loss ile karşılaştırılır. Ceza uzaklık üzerinden verileceği için regression tahminlerinde MSE loss yeterli görülmüştür.

Inference: Çıkarım aşamasında açıklandığı üzere [0.5, 1.0, 1.5] katsayılarıyla multi-scale olarak test verisini ele alınıp 224x224 windowlar üzerinden modele verilmiştir. Model çıktıları sonucunda sliding window yönteminin doğal bir sonucu olarak Non-Max-Suppression kullanılması zorunlu olmuştur.

Tüm bu optimizasyon çalışmalarına rağmen kullanılan yöntemin ve image pyramid'in doğası gereği her inference için binlerce patch için tahmin yürütülmüş ve ortalama 5-6 hatalı tahmin yapılmıştır. Patch başına başarımlar yüksek de olsa bu durum f1 skorlarının ve mAP50 metriğinin çakılmasına sebep olmuştur. Bunun sonucunda yöntem 1 yani sıfırdan bir CNN altyapısını elimizdeki veri seti kullanılarak üreterek hedeflenen mAP50 %70 başarımlar hedefinin uygulanmasının oldukça zor olduğu kanısına varılmıştır.

Deney Sonuçları:

Dene y No	LR	Conv. Block Count	H1	H2	Prec.	F1-Skor	Early stopping at
1	1e ⁻⁴	4	128	64	0.968	0.967	Epoch 13
2	1e ⁻⁴	4	64	32	0.976	0.970	Epoch 24
3	1e ⁻⁴	5	128	64	0.975	0.979	Epoch 15
4	1e ⁻⁴	5	64	32	0.979	0.977	Epoch 17
5	1e ⁻⁵	4	128	64	0.964	0.963	Epoch 26
6	1e ⁻⁵	4	64	32	0.957	0.960	Epoch 21
7	1e ⁻⁵	5	128	64	0.961	0.966	Epoch 17
8	1e ⁻⁵	5	64	32	0.971	0.964	Epoch 13

Değerlendirme:

LR Etkisi: Model 1e-5 öğrenme oranında 1e-4 e göre daha düşük skorlar gösterdiği tespit edilmiştir. Düşük öğrenme oranının ağırlık güncellemelerinde yetersiz kaldığı kanısına varılmıştır.

Convolution Block Sayısı: Her katmanda 2 katına çıkan filtre sayısına sahip convolution blocklarında 5 conv layer ile dense head'e spatial olarak daha iyi anlam çıkartılmış 7x7 conv çıkarımı iletilmesi, 4 conv layer ile 14x14 iletilmesine göre daha yüksek başarımlar göstermiştir.

Dense Head: Daha geniş kapasiteye sahip 128,64 lük dense head 5 convolution layerla olan kombinasyonu öznelikleri yorumlamakta diğer kombinasyonlara göre daha başarılı olmuştur.

Sonuç: Değerlendirmeler sonucunda en başarılı konfigürasyon Deney 3 olarak belirlenmiştir. Optimize edilmiş mimari aşağıda verilmiştir:

Feature Extraction:

- Katman 1 (Conv Block 1):

CNN layer: 32 kanal, 3x3 kernel, zero padding=(1,1), stride=(1,1), Batch Normalization, Leaky ReLU aktivasyonu, Max Pooling (2x2)

- Katman 2 (Conv Block 2):

CNN layer: 64 kanal, 3x3 kernel, zero padding=(1,1), stride=(1,1), Batch Normalization, Leaky ReLU aktivasyonu, Max Pooling (2x2)

- **Katman 3 (Conv Block 3):**

CNN layer: 128 kanal, 3x3 kernel, zero padding=(1,1), stride=(1,1), Batch Normalization, Leaky ReLU aktivasyonu, Max Pooling (2x2)

- **Katman 4 (Conv Block 4):**

CNN layer: 256 kanal, 3x3 kernel, zero padding=(1,1), stride=(1,1), Batch Normalization, Leaky ReLU aktivasyonu, Max Pooling (2x2)

- **Katman 5 (Conv Block 5):**

CNN layer: 512 kanal, 3x3 kernel, zero padding=(1,1), stride=(1,1), Batch Normalization, Leaky ReLU aktivasyonu, Max Pooling (2x2)

Dense Heads:

CNN katmanlarından çıkan 512 kanallı 7x7 boyutundaki uzamsal öznitelik matrisi, tek boyutlu bir vektöre çevrilerek 25.088 nöron fully connected dense katmanlara aktarılmıştır.

- **Katman 6 (Dense Layer 1):**

Fully Connected layer: 128 nöron, Leaky ReLU aktivasyonu, Dropout (p=0.3)

- **Katman 7 (Dense Layer 2):**

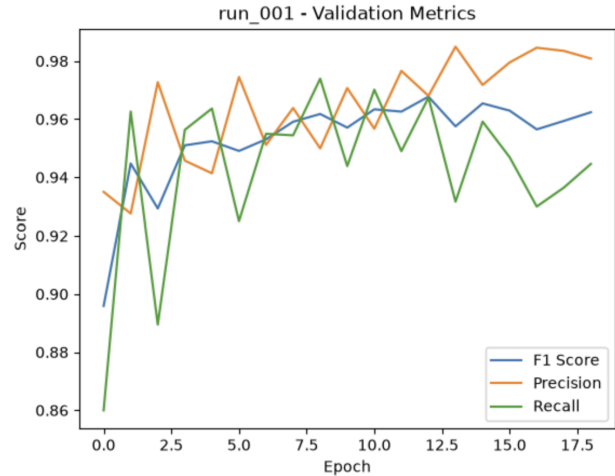
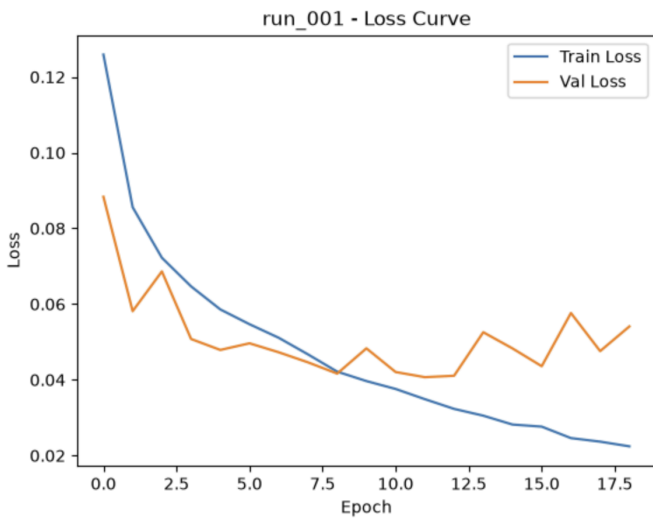
Fully Connected layer: 128 nöron, Leaky ReLU aktivasyonu, Dropout (p=0.3)

Output Layer:

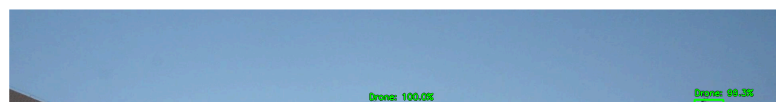
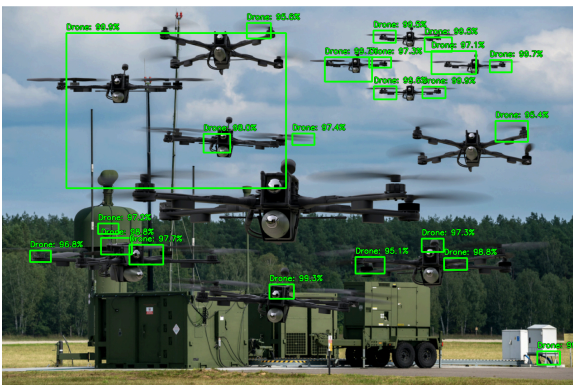
BBox regression'u için MSE loss confidence loss için BCE+Focal Loss kullanılmıştır.

- **Fully Connected Layer:**

5 nöron(1 nöron confidence, 4 nöron bbox regression{x,y,w,h})



Real Life Tests: (İnternette rastgele indirilen drone görüntüleriyle gerçekleştirilen testler)



5. Yöntem 2

Ön işleme

Yöntem 2 de veri setindeki tüm resimler genişliği veya yüksekliği 256 pikseli aşmayacak şekilde en-boy oranını (aspect ratio) koruyarak yeniden boyutlandırılmıştır (resize edilmiştir). Bu ön işlemenin uygulanmasının sebebi veri setindeki büyük resimlerin (örneğin 1500x1000 boyutundaki resimler) GPU'nun hafızasının taşmasına sebep olup eğitimi engellemesidir. Bu sayede GPU'nun hafızasının taşması önlenmiştir. Görüntüler modele verilmeden önce her bir piksel değeri 255.0'e bölünerek 0-1 arasında normalize edilmiştir. Ardından görseller ImageNet'in ortalama ve standart sapmasına göre normalize edilmiştir (Her bir RGB kanalındaki pikselden, o kanala özgü ortalama değer (mean) çıkarılmış, ardından o kanal özgü standart sapmaya bölünmüştür). Bu uygulamanın sebebi, yöntemde kullanılacak olan ResNet18'in eğitim verisine aynı ön işlemenin uygulanmış olmasıdır.[6] [7]

Yöntem açıklaması

Bu yöntemde ResNet18'in tüm CNN layer'larını aktarmalı öğrenme kapsamında omurga (backbone) olarak kullanıp hazır katmanların üstüne kendi tespit katmanlarını (detection head) ekledim. Yöntem 2'de amacımız Fully Convolutional neural network tasarlamaktır, bu sebeple eklenen tespit katmanlarının tamamı CNN katmanları içermektedir. Mimari içerisinde Fully Connected katmana yer verilmemiştir.

Fully Convolutional Neural network mimarisi sayesinde model windowing uygulamaya gerek duymamaktadır. Bunun yerine resmi 32x32 boyutundaki grid'lere (ızgaralara/hücrelere) bölerek her bir grid (ızgara/hücre) için bbox konumu (bbox_merkezi_x, bbox_merkezi_y, bbox_genişliği, bbox_yüksekliği) ve grid merkezinde drone olma ihtimali (positive class probability) için tahminde bulunuyor. Problemimiz tek sınıflı olduğu için her grid için ayrıyeten bir objectness değeri üretmeye gerek duyulmamaktadır. Bunun yerine drone olma ihtimali (positive class probability) objectness skoru olarak işlev görmektedir. Modelin tasarımı, veri seti de göz önüne alınarak her bir grid'e en fazla bir obje düşebileceği varsayımı altında yapılmıştır ve her bir grid için en fazla bir drone tespit edilebilmektedir.

Fully Convolutional neural network mimarisinin avantajlarından bir diğeri ise girdi görüntüsünün (input image) herhangi bir boyutta olabilmesi ve yeniden boyutlandırılmaya ihtiyaç duymamasıdır. Tasarlanan model yığınlama (batching) uygulanması takdirinde bir yığındaki (batch'deki) görüntülerin uzamsal boyutlarının (spatial dimensions) uyuşması için görüntülere padding eklenmektedir. Omurga (backbone) olarak ResNet18 kullanıldığı için görüntü hazır ResNet18 katmanları tarafından 32 kat küçültülmektedir [6]. Izgara (grid'lerin) çakışmaması için ve olası hatalardan sakınmak için modelin boyutları 32 ye tamamlanacak şekilde padding eklenmektedir. Padding pikselleri maskelenmekte ve kayıp fonksiyonuna (loss function) etki etmemektedir.

Eğitim esnasında modele verilen her bir grid (ızgara) için etiket (ground truth label) verilmiştir. Eğer objenin merkezi ızgara içerisinde ise o ızgara içerisinde model var olarak etiketlenmiştir. Objeye içeren ızgaralar için bbox (bounding box) etiketleri verilirken bbox'ın merkez koordinatlarının ızgara içerisindeki göreceli konumu verilmiştir. Bbox'ın genişliği ve yüksekliği ise ızgaranın boyutlarına

bölünmüştür, bunun sebebi bbox boyutlarının değişken boyutlu olmasından kaynaklı olarak kayıp fonksiyonunda (loss function) dengesiz bir yükselme yapmasını engellemektir.

Early stop: Patience değeri 3 olarak belirlenmiştir. Model 3 epoch boyunca training loss'u düşüremezse eğitim durur.

ResNet18'in son katmanlarını fine tune etmediğim durumlarda tek aşamada yalnızca tespit katmanlarını eğittim ve early stop uyguladım.

ResNet18'in son katmanlarını fine tune ettiğim durumda 2 aşamalı eğitim uyguladım.

1. Aşamada early stop olmadan yalnızca tespit katmanlarını eğittim ve ResNet18'i dondurdum.
2. Aşamada early stop mekanizmasını aktif ettim ve ResNet18'in son 2 katmanını tespit katmanıyla beraber fine tune ettim.

Model Mimarisi:

ResNet18'in tüm CNN katmanları (Aktarmalı Öğrenme ile Alınan Önceden Eğitilmiş Katmanlar) [6]:

Alınan hazır modelin mimarisinde herhangi bir değişiklik yapılmamıştır.

Tespit Katmanları (Hiperparametre Optimizasyonu Sonucu Bulunmuş Optimal Mimari):

1. Katman 1:

CNN layer: 256 nöron , 3x3 kernel, zero padding=(1,1), stride=(1,1), ReLU

2. Katman 2:

CNN layer: 128 nöron , 3x3 kernel, zero padding=(1,1), stride=(1,1), ReLU

3. Katman 3:

CNN layer: 64 nöron, 3x3 kernel, zero padding=(1,1), stride=(1,1), ReLU

4. Çıktı Katmanı:

CNN layer: 5 nöronun , 1x1 kernel, valid padding, stride=(1,1)

3 tane nöronun (drone sınıfını tahmin eden nöronun ve bbox'ın merkezinin x ve y koordinatlarını tahmin eden 2 nöronunun) çıktıları sigmoid fonksiyonuna verilmiştir. Bunun sebebi bu 3 tahmininin 0-1 arasında değer almasını sağlamaktır.

2 tane nöronun (bbox'ın genişliğini ve yüksekliğini tahmin eden 2 nöronun) çıktıları ise softplus fonksiyonuna verilmiştir. Softplus fonksiyonun tercih edilmesinin sebebi bbox genişlik ve yükseklik tahminlerinin pozitif değer almasını sağlamaktır.

Kayıp fonksiyonu:

Modelin kayıp fonksiyonu sınıflandırma kaybının ve lokalizasyon kaybının toplamından elde edilmektedir.

Sınıflandırma kaybını hesaplamak için Binary Cross Entrophy fonksiyonu kullanılmaktadır. Sınıflandırma problemimizin ikili sınıflandırma olması sebebiyle tercih edilmiştir.

Lokalizasyon kaybı (localization loss), modelin bbox koordinatlarını kötü tahmin ettiği zaman modeli cezalandıran kayıp fonksiyonudur. Lokalizasyon kaybı için L1 loss, MSE, Smooth L1 Loss fonksiyonları hiperparametre kombinasyonu aşamasında denenmiştir.

```
----- MSE Loss -----
val_precision    0.547747
val_recall       0.4151
val_f1           0.472287
val_mAP50        0.302725
Name: 9, dtype: object

----- L1 Loss -----
val_precision    0.594526
val_recall       0.442356
val_f1           0.507275
val_mAP50        0.341661
Name: 2, dtype: object

----- smooth L1 Loss -----
val_precision    0.526096
val_recall       0.394737
val_f1           0.451047
val_mAP50        0.282319
Name: 3, dtype: object
```

Yukarıda verilen sonuçlarda görüldüğü üzere lokalizasyon kaybı için en ideal fonksiyon L1 loss fonksiyonudur.

En kötü sonuçları Smooth L1 loss fonksiyonunun verdiği gözlemlenmiştir.

Tespit katmanları için aktivasyon fonksiyonlarının seçimi:

Ara katmanlarda uygulanan aktivasyon fonksiyonu:

Çıktı katmanları haricinde kullanılacak aktivasyon fonksiyonunu seçerken ReLU ve Leaky ReLU fonksiyonları denenmiştir. Bunun sebebi geri yayılım (backpropagation) aşamasında sigmoid fonksiyonunun kaybolan gradyent (vanishing gradient) sorununa sebep olma ihtimalidir. Bunun beraberinde tanh ve SiLU fonksiyonları, ReLU ve Leaky ReLU'ya kıyasla daha fazla hesap gerektiren (computationally expensive) fonksiyonlar olması sebebiyle tercih edilmemiştir. Leaky ReLU ve ReLU aktivasyon fonksiyonları hiperparametre optimizasyonu esnasında ayrı deneylerde denenmiş olup ReLU'nun daha iyi sonuç verdiği gözlemlenmiştir.

```
----- experiment 7 -----
val_precision    0.526096
val_recall       0.394737
val_f1           0.451047
val_mAP50        0.282319
Name: 3, dtype: object
----- experiment 8 -----
val_precision    0.481277
val_recall       0.354323
val_f1           0.408156
val_mAP50        0.228987
Name: 3, dtype: object
```

Experiment 7’de ara katmanlarda aktivasyon fonksiyonu olarak ReLU kullanılmıştır.

Experiment 8’de ara katmanlarda aktivasyon fonksiyonu olarak Leaky ReLU kullanılmıştır.

Aktivasyon fonksiyonu haricindeki hiperparametreler sabittir.

Yukarıda görüldüğü üzere ölçülen her bir metrikte ReLU daha iyi sonuç vermiştir.

Bu sonuç Leaky ReLU’nun oluşturduğu negatif ağırlıkların modelin öğrenmesine olumsuz etki ettiğini göstermektedir.

Çıktı katmanında uygulanan aktivasyon fonksiyonları:

Çıktı katmanlarında pozitif sınıf (drone bulunuyor mu tahmini / positive class probability) tahmini yapan nöronun çıktısı olasılık değeri üretmesi gerektiğinden sigmoid fonksiyonuna verilerek 0-1 arasına sıkıştırılmıştır.

Eğitim sırasında modele verilen bbox etiketleri (bounding box labels) verilirken modelin öğrenmesini kolaylaştırması açısından bbox’ın merkezini veren x koordinatı ızgara (grid) genişliğine, y koordinatı ise ızgara (grid) yüksekliğine bölünerek 0 ile 1 arasında normalize edilmiştir. x ve y değerlerinin ızgara (grid) boyutuna bölünmesinin sebebi her bir ızgaranın merkezi kendi içerisinde kalan kalan objenin koordinatlarını öğrenmesinin istenmesidir. Bu sebeple bbox merkezinin x ve y koordinatlarının tespit eden 2 nöronun çıktısı da 0-1 arası değer vermesi için sigmoid fonksiyonuna verilmiştir.

Bbox’ın genişliği ve yüksekliği negatif değer alamayacağı için bbox’ın genişliğini ve yüksekliğini tahmin eden nöronların çıktıları softplus fonksiyonuna verilerek negatif değer almaları engellenmiştir.

Başarım Metriklerinin Hesaplanması:

TP: True positive

FP: False positive

FN: False positive

Model tahminleri pozitif sınıf tahminine göre belirlenen objectness threshold'unun üstünde ise pozitif tahmin kabul edilir.

Pozitif tahminler pozitif sınıf tahminine göre en yüksekten en düşüğe sıralanır. Sırasıyla öncelikle daha yüksek objectness (sınıf tahmini) skoruna sahip tahminler eğer IoU değeri %50'den yüksek ise daha önce daha yüksek bir tahminle eşleşmemiş objeler arasından en yüksek IoU'ya sahip olunan objenin bbox'ı ile eşleştirilir. Herhangi bir objeyle eşleşen tahminler TP sayılmaktadır. Eğer IoU %50'den düşükse veya herhangi bir tahmin herhangi bir objeyle eşleşmediyse tahmin FP sayılır. Eğer herhangi bir tahminle eşleşmemiş bir obje varsa bu obje FN sayılır. TN (True negative) metriğinin bizim problemimizde anlamsız olması ve ölçümünün kullanılan yöntemde mümkün olmaması sebebiyle ölçülememiştir.

Bu sebeple accuracy skoru ölçülememektedir. Modelin başarımları başta mAP50 ve F1 skordur. Bunlarla beraber precision ve recall ölçülmüştür.

Hiperparametre Optimizasyonu:

Her bir hiperparametre kombinasyonu için TP, FP, FN, Precision, Recall, F1 skor, mAP50 metrikleri hesaplanmıştır.

Birincil başarımlarımız mAP50 değeridir. Bununla beraber deneylerimizde de gözlemlediğimiz üzere f1 skoru, mAP50 ile beraber artış göstermektedir. İdeal objectness threshold'unun belirlenmesi adına f1 skoru da ikincil başarımlarımız olarak belirlenmiştir.

Her bir hiperparametre kombinasyonunun başarımları farklı objectness threshold'lar ile ölçülmüştür. Bir kombinasyonun başarımları, denenen threshold'lar arasındaki en yüksek f1 başarımlarına sahip threshold'un başarımları olarak alınmıştır. mAP50 skoru her bir threshold değeri için recall-precision eğrisi üzerinden hesaplanması sebebiyle threshold değerinden bağımsız bir metriktir. Bu yöntemle modeller arasından en yüksek f1 skor ve mAP50'ye sahip model seçilmiştir.

TK1: Tespit katmanı 1 convolutional nöron sayısı

TK2: Tespit katmanı 2 convolutional nöron sayısı

TK3: Tespit katmanı 3 convolutional nöron sayısı

LR: Learning Rate

DR: Dropout Rate

Bbox Loss: Localization loss'ünü hesaplamak için kullanılan kayıp (loss) fonksiyonu

BLC (Bbox loss coefficient): Localization kaybını, sınıflandırma kaybı ile toplayıp toplam loss'u bulurken lokalizasyon kaybının çarpıldığı katsayı

Aktiv. Fonk: Tespit katmanında kullanılan Aktivasyon Fonksiyonu

FT: Fine tune edilen ResNet18 katman sayısı

Deney No	LR	Bbox Loss	DR	Aktiv. Fonk.	BLC	FT	Optimizer	TK1	TK2	TK3	mAP50	Precision	Recall	F1
1	0,001	Smooth L1	0	ReLU	1	0	Adam	256	128	64	0,06	0,27	0,17	0,21
2	0,001	Smooth L1	0	ReLU	1	0	Adam	256	128	64	0,07	0,32	0,17	0,22
3.1	0,001	Smooth L1	0	ReLU	1	0	Adam	256	128	-	0,058	0,26	0,16	0,20
3.2	0,001	Smooth L1	0	ReLU	1	0	Adam	256	-	-	0,054	0,23	0,13	0,17
4.1	0,001	Smooth L1	0	ReLU	1	2	Adam	256	128	64	0.19	0.43	0.31	0.36
4.2	0,001	Smooth L1	0	ReLU	1	1	Adam	256	128	64	0.16	0.43	0.30	0.35
5.1	0,001	Smooth L1	0.3	ReLU	1	0	Adam	256	128	64	0.07	0.32	0.17	0.227
5.2	0,001	Smooth L1	0.5	ReLU	1	0	Adam	256	128	64	0.10	0.36	0.17	0.237
7	0,001	Smooth L1	0	ReLU	5	2	AdamW	256	128	64	0.28	0.52	0.39	0.45
8	0,001	Smooth L1	0	Leaky ReLU	5	2	AdamW	256	128	64	0.22	0.48	0.35	0.40
9.1	0,001	L1	0	ReLU	5	2	AdamW	256	128	64	0.34	0.59	0.44	0.50
9.2	0,001	MSE	0	ReLU	5	2	AdamW	256	128	64	0.30	0.54	0.41	0.47

Not: Deney 1 ve Deney 2 arasındaki fark maksimum epoch sayısının deney 1 de 60 iken deney 2 de 120'ye çıkarılması ve geri kalan tüm deneylerde 120 olarak devam ettirilmesidir. İstisnai olarak son deneyde 160 epoch denenmiştir.

Not: Deney isimlendirmeleri proje 2 dosyasındaki experiment_x.json dosyalarına göre isimlendirilmiştir. İsimlendirme <dosya-adı>.<kombinasyonun_dosyadaki_sırası> şeklindedir.

Not: Her bir deney sonucu .json formatında Yontem2 klasörü içerisinde bulunmaktadır. Bununla beraber validation.ipynb içerisinde deneylerle beraber grafiklere de yer verilmiştir. Deney sayısının fazla olması sebebiyle önemli bulunan deneyler tabloya kaydedilmiştir.

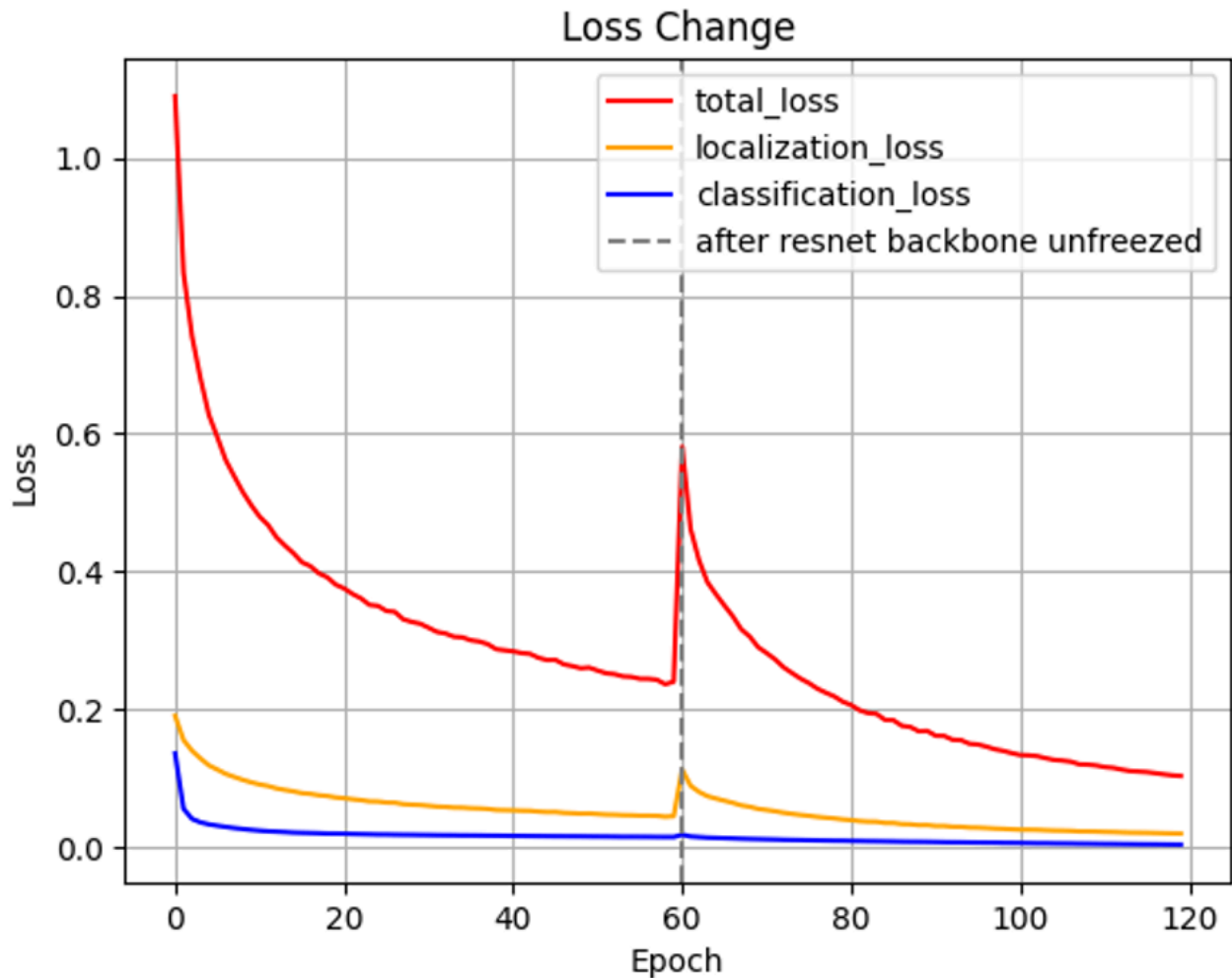
İlk olarak modeli ResNet18 katmanlarını tamamen dondurarak eğittim.

2, 3 ve 4 adet tespit katmanı sırayla denenerak tespit katmanları için optimal derinlik 4 katman olarak bulunmuştur.

Dropout katmanı ResNet18 ile tespit katmanları arasına konmuştur. %0, %30, %50 dropout oranında en başarılı oran %0 bulunmuş ve dropout uygulanmamasına karar verilmiştir. (Deney 2 ve Deney 5)

Learning rate için 0.01, 0.001, 0.0001 değerleri denenmiş, ideal değer olarak 0.0001 seçilmiştir.

Daha sonraki aşamalarda daha iyi bir performans sergilemesi sebebiyle eğitimi 2 aşamalı yapmaya başladım. 1. Aşamada ResNet18 katmanları dondurulup tespit katmanlarını eğittim. 1. Aşamada early stop uygulamadım. 2. aşamada ResNet18'in son 2 katmanını unfreeze edip (öğrenmeye açıp) tespit katmanları ile beraber fine tune etti. Yapılan deneyler sonucu ResNet18'in son 2 katmanı fine tune etmenin ResNet18'in sadece son katmanını fine etmek veya ResNet18'i hiç fine etmemekten daha iyi sonuç verdiği gördüm. 2. Aşamada early stop uygulanmıştır.



2 aşamalı eğitimde esnasında ResNet18'in son 2 katmanı eğitime başlandığı epoch'ta loss fonksiyonunda ani bir artış gözlemlenmiştir. Bunun sebebinin tüm katmanlar için aynı optimizör kullanılması olduğu varsayıp aynı deney farklı ResNet18 katmanlarında ve tespit katmanlarında farklı bir optimizör kullanılarak tekrarlanmıştır. Bu deneyde ResNet18'in optimizör'üne learning rate olarak 0.00001, detection head'e ise önceki deneydeki gibi 0.0001 verilmiştir. Bunun sebebi ResNet18 katmanlarının daha ince fine tune edilmesinin istenmesidir. Bununla early stop mekanizması aktif olmasına rağmen model erken durmadığı için max epoch sayısı 160'a artırılmıştır. Bunun sonucunda eğitim metrikleri artmış fakat geçerleme metrikleri düşmüştür ve modelde overfitting gözlemlenmiştir.

ReLU ve Leaky ReLU karşılaştırılmış ve ReLU daha başarılı bulunmuştur. (Deney 7 ve Deney 8)
Adam optimizör ile AdamW karşılaştırıldığında AdamW'nin Adam'a kıyasla kayda değer bir başarımlı artışı sergilediği gözlemlenmiştir. (Deney 4.2 ve Deney 7) [8]

Geçerleme sonucu bulunan en iyi mimari model mimarisi başlığı altında yukarıdaki sayfalarda belirtilmiştir.

Mimariye ek olarak bulunan optimal hiperparametreler:

Optimizör: AdamW

Learning rate:0.0001

İki aşamalı eğitim:

İlk aşamada 60 epoch tespit katmanı eğitimi

İkinci aşamada maksimum 60 epoch ResNet18 son 2 katmanı tespit katmanı ile beraber eğitilmeli.

Lokalizasyon fonksiyonu için L1 loss kullanılmalı, total loss'a eklenmeden 5 ile çarpılması lokalizasyon hatalarının azaltılmasını ve model başarısının artmasını sağlamıştır.

Dropout uygulanmamalı.

Aktivasyon fonksiyonu olarak ReLU kullanılmalı.

İdeal objectness threshold son model için 0.5 olarak tespit edilmiştir.

Test sonucu

```
===== Test Results =====  
Test TP: 1065  
Test FP: 446  
Test FN: 965  
  
Test precision: 0.7048312375909993  
Test recall: 0.5246305418719212  
Test f1: 0.6015249929398474  
  
Test mAP50: 0.4771386384963989
```

6. Deney Sonuçları ve Değerlendirme

İki yöntem için deney sonuçları ve değerlendirmeleri ayrı ayrı Yöntemler kısımlarında verilmiştir.

7. Sonuç ve Potansiyel Geliştirmeler

Bu bölümde, drone tespiti üzerine önerdiğimiz iki ayrı yöntem üzerinden öncelikle ayrı ayrı geliştirilen modellerin son durumları ve potansiyel geliştirmeleri ardından iki yöntemin birbiriyle karşılaştırılması sunulmuştur.

Yöntem 1:

Son Durum Zafiyetleri:

- **Yüksek FP Rate:** Geliştirilen CNN windowing çözümünde, windowing yönteminin gerektirdiği ölçüde model başarımlarını artırmış test verisinde başarımların mAP@50 de yüzde 17 seviyesini geçememiştir.
- **Sürü Drone'larda NMS sönümlenmesi:** Görüntüde birbirine çok yakın uçan veya sürü (swarm) halinde bulunan dronelar üzerinde yapılan testlerde, Non-Maximum Suppression (NMS) algoritmasının agresif çalıştığı gözlemlenmiştir. NMS_Threshold yükseltmesi de anchor box şeklinin tek çeşit olmasından kaynaklı farklı sorunlara yol açtığı için bu soruna da bir çözüm bulunamamıştır.
- **Drone renk doku genellemesinde eksiklik:**
Drone renk ve dokularının çok çeşitli olabilmesinden kaynaklı ör. bembeyaz olan veya üzerinde kargo taşıyan drone'lar model tarafından confidence'ı düşük tutulmaktadır. Sorun veri seti kaynaklı olduğu için bu duruma da bir çözüm bulunamamıştır.

Potansiyel Geliştirme Önerileri:

- **Anchor Box Kullanımı:**
NMS sönümlenmesi problemini ve sürü tespitindeki zafiyeti çözmek amacıyla, modele tek bir pencere çıktısı üretmek yerine farklı boyut ve en-boy oranlarına sahip Anchor Box'lar denenmesi önerilmektedir.
- **Gray-Scale Fine-Tuning:**
Veri seti renk doku eksikliğini gidermek modele bu kapsamda generalization yeteneği sunmak için aynı veri seti gray-scale'e çevrilip ilk katman öncesine 3 filterlı bir CNN layer preprocess eklenip model girdi mimarisini değiştirilmeden pseudo RGB veriler ile modelin fine-tune edilmesi önerilmektedir.
Veya daha straight-forward bir çözüm olarak preprocess işlemine kapsamlı bir color augmentation eklenebilir.
- **Dynamic Tiling: [5]**
Kayan pencere algoritmasının getirdiği yüksek işlem yükünü ve ürettiği arka plan gürültüsünü (False Positive) azaltmak için, sabit ölçekleme katsayıları yerine, çekim yapılan irtifaya veya nesne yoğunluğuna bağlı olarak tarama penceresini dinamik olarak ayarlayan adaptif çıkarım tekniklerine (örn. SAHI) geçiş yapılması sistemin verimliliğini artıracaktır.

Yöntem 2:

Son Durum Zafiyetleri:

- Model donanım yetersizliğinden dolayı resize edilmiş küçük boyutlu (boyutu 256x256'yı aşmayan) resimlerle eğitilmiştir.
- Drone'ların şekil ve renklerinin çeşitliliğinden dolayı bazı drone'lar tanınmamaktadır.

Potansiyel Geliştirme Önerileri:

- Model daha yüksek bir donanımda farklı boyutlardaki görüntülerle bellek kısıtı olmadan eğitilebilir.
- Quantization uygulanarak daha büyük görüntülerle bellek sorunu yaşanması önlenabilir.
- Pipeline parallelism ile dağıtık sistem model eğitilerek daha büyük boyutlardaki görüntülerle eğitim esnasında bellek taşması önlenabilir.
- Daha çeşitli drone'ların tanınması için daha büyük bir dataseti kullanılabilir.

Model kuş, balon gibi uçan objeleri yanlış tanımaması ve hem küçük hem büyük objeleri tespit edebilmektedir. Buna rağmen drone'ları tespit edemediği örneklerle de rastlanmıştır.



Yöntemlerin Karşılaştırılması:

Yöntem 1, sıfırdan tasarlanmış bir CNN ile Geleneksel Kayan Pencere (Sliding Window) ve Görüntü Piramitleri yaklaşımını benimserken; Yöntem 2, Aktarmalı Öğrenme (Transfer Learning) tabanlı ResNet-18 backbone kullanan ve grid mantığıyla çalışan Fully Convolutional Network - Gridlere ayırma yaklaşımını kullanmıştır.

Inference (çıkarım) Hızı:

Yöntem 1'de kullanılan Sliding Window mimarisi sebebiyle tek bir görsel üzerinden çıkarım yapılması için çok sayıda window modele girdi olarak verilmesi gerekmektedir. Bu inference öncesi preprocessing işlemi modelin çıkarım yapmasını oldukça yavaşlatmaktadır.

Yöntem 2’de ise inference aşamasının sliding window kullanmayıp girdi resmini bir bütün olarak tek aşamada alması, Yöntem 2’yi inference hızı açısından Yöntem 1’in önüne geçirmektedir.

Aynı obje için modelin birden fazla bbox üretmesi sorununa karşı çözümler:

Yöntem 1’de aynı obje için birden fazla bbox üretilmesini engellemek amacıyla test sonrasında non-maximum-suppression ile post-processing uygulanmıştır.

Yöntem 2’de aynı obje için birden fazla bbox üretilmesinin önüne geçmek amacıyla model eğitilirken yalnızca objenin merkezinin bulunduğu ızgarayı (grid) pozitif sınıfla etiketleyecek şekilde eğitilmiştir. Eğitim esnasında model objenin merkezinin bulunduğu ızgarayı tespit etmediği takdirde loss function artmakta ve model cezalandırılmaktadır.

8. Kaynakça

- [1] Hobden P, Srivastava S and Nurellari E (2022) FPGA-Based CNN for Real-Time UAV Tracking and Detection. *Front. Space Technol.* 3:878010. doi: 10.3389/frspt.2022.878010
- [2] H. Lee et al., "CNN-Based UAV Detection and Classification Using Sensor Fusion," in *IEEE Access*, vol. 11, pp. 68791-68808, 2023, doi: 10.1109/ACCESS.2023.3293124.
- [3] X. Di, K. Cui, and R.-F. Wang, "Toward Efficient UAV-Based Small Object Detection: A Lightweight Network with Enhanced Feature Fusion," *Remote Sensing*, vol. 17, no. 13, p. 2235, 2025, doi: 10.3390/rs17132235.
- [4] J. Ni, S. Zhu, G. Tang, C. Ke, and T. Wang, "A Small-Object Detection Model Based on Improved YOLOv8s for UAV Image Scenarios," *Remote Sensing*, vol. 16, no. 13, p. 2465, 2024, doi: 10.3390/rs16132465.
- [5] "Maritime Small Object Detection from UAVs using Deep Learning with Altitude-Aware Dynamic Tiling," *arXiv preprint*, arXiv:2511.19728, 2025. URL: <https://arxiv.org/html/2511.19728v1>
- [6] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778
- [7] J. Deng, W. Dong, R. Socher, L. -J. Li, Kai Li and Li Fei-Fei, "ImageNet: A large-scale hierarchical image database," 2009 IEEE Conference on Computer Vision and Pattern Recognition, Miami, FL, USA, 2009, pp. 248-255
- [8] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *arXiv preprint* arXiv:1711.05101, 2019. doi: 10.48550/arXiv.1711.05101